

1. Apresentação do Projeto

1.1 O que é este projeto

Este projeto é uma continuação de um trabalho anterior em que os dados de um controle de fluxo financeiro estavam organizados em planilhas Excel e eram utilizados em um dashboard no Power BI.

Na primeira versão do projeto, o foco estava mais na visualização dos dados. Porém, durante o desenvolvimento, surgiu o interesse em entender melhor como os dados poderiam ser organizados em um banco de dados e como ferramentas de programação poderiam ajudar nesse processo.

Por esse motivo, foi desenvolvido este novo projeto, que busca reorganizar os mesmos dados utilizando SQL Server para armazenar as informações e Python para automatizar a leitura e inserção dos dados no banco.

A ideia principal não foi criar um sistema complexo, mas sim aprender na prática como funciona o processo de estruturação de dados utilizando banco de dados e scripts simples de automação.

1.2 Objetivos do Projeto

O desenvolvimento deste projeto tem como objetivo principal colocar em prática conceitos fundamentais relacionados ao trabalho com dados estruturados.

Entre os objetivos específicos do projeto estão:

- **estruturar os dados em um banco de dados relacional no SQL Server;**
- **compreender o processo de criação e organização de tabelas em um banco de dados;**
- **utilizar Python para automatizar a leitura de arquivos Excel e a inserção dos dados no banco;**
- **praticar consultas SQL para explorar e relacionar as informações armazenadas;**
- **entender, de forma prática, como diferentes ferramentas podem ser utilizadas em conjunto dentro de um fluxo de dados.**

A intenção não foi construir uma solução complexa, mas sim desenvolver uma implementação funcional que permita compreender cada etapa do processo.

1.3 Escopo do Projeto

Este projeto tem como foco principal a construção de um fluxo simples de organização e carga de dados utilizando Excel, SQL Server e Python.

Dentro desse escopo, foram realizadas as seguintes etapas:

- **criação da estrutura inicial do banco de dados no SQL Server;**
- **definição das tabelas responsáveis por armazenar os dados;**
- **leitura dos arquivos Excel que contêm as informações originais;**
- **inserção automatizada desses dados no banco utilizando Python;**
- **realização de consultas SQL para verificar e explorar os dados inseridos.**

2. Justificativa do Projeto

2.1 Problema Identificado

No projeto anterior, os dados utilizados estavam organizados em diferentes arquivos Excel. Esses arquivos continham informações como bancos, plano de contas, movimentações e saldo anterior.

Embora essa estrutura funcionasse para a construção inicial do dashboard no Power BI, trabalhar apenas com planilhas apresentava algumas limitações. Cada atualização dependia da manutenção manual dos arquivos, e qualquer alteração nas planilhas poderia impactar diretamente o funcionamento do relatório.

Além disso, durante o desenvolvimento do projeto anterior surgiu a curiosidade de entender como esses dados poderiam ser organizados dentro de um banco de dados, já que em muitos cenários profissionais as informações não ficam armazenadas apenas em planilhas. Essa situação motivou a criação de uma nova versão do projeto, com o objetivo de reorganizar os dados utilizando ferramentas mais voltadas para manipulação e armazenamento estruturado de dados.

2.2 Solução Proposta

Para lidar com essa situação, foi decidido reconstruir parte do projeto utilizando uma estrutura baseada em banco de dados.

A ideia foi utilizar o **SQL Server** para criar as tabelas responsáveis por armazenar os dados de forma organizada. Dessa forma, as informações deixariam de depender exclusivamente dos arquivos Excel e passariam a estar centralizadas em um banco de dados.

Além disso, foi utilizado **Python** para automatizar a leitura das planilhas e realizar a inserção dos dados nas tabelas criadas no **SQL Server**. Isso permite que o processo de carregamento dos dados seja feito de forma mais simples e repetível.

Com essa abordagem, o projeto passou a ter um fluxo mais estruturado:

- **Os dados continuam sendo mantidos em arquivos Excel como fonte inicial;**
- **OSQL Server é utilizado para armazenar e organizar essas informações;**
- **O Python é responsável por realizar a carga dos dados no banco.**

Essa solução foi escolhida principalmente como forma de aprendizado, permitindo entender na prática como diferentes ferramentas podem trabalhar juntas em um processo de tratamento de dados.

3. Arquitetura da Solução

Dentro da estrutura do banco de dados, foi adotada uma etapa intermediária chamada tabelas **staging**.

As tabelas staging funcionam como uma área inicial onde os dados são carregados no banco exatamente como vêm das planilhas Excel. Nesse estágio, os dados ainda não passam por transformações mais complexas.

A decisão de utilizar essa etapa foi tomada por dois motivos principais.

O primeiro motivo é organização do processo de carga. Ao inserir os dados primeiro nas tabelas staging, fica mais fácil separar o momento em que os dados são carregados do momento em que eles são transformados ou organizados para análise.

O segundo motivo é facilitar o tratamento dos dados utilizando SQL. Depois que os dados estão armazenados nas tabelas staging, é possível utilizar consultas SQL para limpar, transformar e organizar essas informações antes de utilizá-las em outras tabelas.

Esse processo permite dividir o fluxo do projeto em etapas mais claras:

- **criação da estrutura inicial do banco de dados;**
- **carga dos dados brutos nas tabelas staging utilizando Python;**

- transformação e organização dos dados utilizando SQL;
- utilização desses dados no Power BI para análise.

4. Documentação Técnica

4.1 Criação do Banco de Dados

```
CREATE DATABASE FluxoCaixaDW;
```

Explicação do que foi feito:

Nesta etapa foi criado um banco de dados no SQL Server chamado FluxoCaixaDW. Esse banco foi criado para armazenar todas as tabelas utilizadas no projeto.

A criação de um banco de dados separado foi necessária para manter os dados do projeto organizados e evitar misturar essas informações com outras bases que possam existir no servidor.

Explicação técnica:

O comando CREATE DATABASE é utilizado no SQL Server para criar uma nova base de dados dentro do sistema de gerenciamento de banco de dados.

Ao executar esse comando, o SQL Server cria automaticamente toda a estrutura necessária para armazenar tabelas, índices e outros objetos que serão utilizados posteriormente no projeto.

Depois da criação do banco, ele passa a ser o ambiente onde serão criadas as tabelas utilizadas para armazenar os dados carregados a partir das planilhas Excel.

Motivo da decisão:

A decisão de criar um banco de dados próprio para o projeto foi tomada para estruturar melhor o ambiente de desenvolvimento. Dessa forma, todas as tabelas, consultas e testes realizados ficam centralizados em um único local.

4.2 Criação das Tabelas Staging

Código utilizado

```

CREATE TABLE stg_Bancos (
    Banco_ID INT,
    Banco NVARCHAR(50)
);

CREATE TABLE stg_Contas (
    Subgrupo_ID INT,
    Subgrupo NVARCHAR(100),
    Conta_ID INT,
    Conta NVARCHAR(100)
);

CREATE TABLE stg_Movimentos (
    Tipo NVARCHAR(20),
    Conta NVARCHAR(100),
    Data DATETIME,
    Banco NVARCHAR(50),
    Valor DECIMAL(18,2)
);

CREATE TABLE stg_SaldoAnterior (
    Banco_ID INT,
    Valor DECIMAL(18,2)
);

```

Explicação do que foi feito

Após a criação do banco de dados, foram criadas quatro tabelas chamadas **tabelas staging**. Essas tabelas têm como objetivo receber os dados diretamente das planilhas Excel utilizadas no projeto.

As tabelas criadas foram:

- **stg_Bancos** – armazena os bancos cadastrados
- **stg_Contas** – armazena as contas e seus subgrupos
- **stg_Movimentos** – armazena os registros de movimentações financeiras
- **stg_SaldoAnterior** – armazena o saldo inicial de cada banco

Essas tabelas funcionam como uma área intermediária onde os dados são carregados inicialmente antes de serem utilizados em consultas ou transformações.

Explicação técnica

O comando CREATE TABLE é utilizado para criar tabelas dentro do banco de dados. Cada tabela possui colunas que armazenam diferentes tipos de informação.

Neste projeto foram utilizados alguns tipos de dados importantes:

INT

Usado para armazenar valores numéricos inteiros, como identificadores de bancos ou contas.

NVARCHAR

Utilizado para armazenar textos, como nomes de bancos, contas e tipos de movimentação. O uso de NVARCHAR permite armazenar caracteres especiais, o que é importante em sistemas que utilizam acentuação.

DATETIME

Usado para armazenar informações de data e hora. Nesse caso, ele registra a data das movimentações financeiras.

DECIMAL(18,2)

Utilizado para armazenar valores monetários. Esse tipo permite números com casas decimais, sendo adequado para representar valores financeiros com precisão.

Motivo da decisão

A criação das tabelas staging foi uma decisão importante para organizar melhor o fluxo de dados do projeto.

Como os dados estavam originalmente em planilhas Excel, foi necessário criar uma estrutura no banco de dados que pudesse receber essas informações antes de qualquer tratamento ou análise.

Ao utilizar tabelas staging, o processo fica dividido em etapas:

- **Os dados são extraídos das planilhas Excel;**
- **O Python carrega essas informações nas tabelas staging;**
- **Depois os dados podem ser consultados ou transformados utilizando SQL.**

Essa abordagem ajuda a separar o momento em que os dados são carregados do momento em que eles são utilizados para análise. Mesmo sendo um projeto simples, essa estrutura segue um padrão comum em projetos de dados

4.3 Carregamento dos Dados do Excel para o SQL Server com Python

Código utilizado

```
import pandas as pd
from sqlalchemy import create_engine

conn_str = (
    "mssql+pyodbc://DESKTOP-G854HDJ/FluxoCaixaDW?"
    "driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes"
)
engine = create_engine(conn_str)

caminho = r"C:\Users\Paulo\Downloads\Projeto_Fluxo_Caixa_SQL_Python\dados_originais"

df_bancos = pd.read_excel(f"{caminho}\\Bancos.xlsx", sheet_name="Bancos")
df_contas = pd.read_excel(f"{caminho}\\PlanoContas.xlsx", sheet_name="PlanoContas")
df_movimentos = pd.read_excel(f"{caminho}\\Movimentos.xlsx", sheet_name="Movimentos")
df_saldo = pd.read_excel(f"{caminho}\\SaldoAnterior.xlsx", sheet_name="SaldoAnterior")

df_bancos.to_sql("stg_Bancos", engine, if_exists="replace", index=False)
df_contas.to_sql("stg_Contas", engine, if_exists="replace", index=False)
df_movimentos.to_sql("stg_Movimentos", engine, if_exists="replace", index=False)
df_saldo.to_sql("stg_SaldoAnterior", engine, if_exists="replace", index=False)
```

Explicação do que foi feito:

Nesta etapa foi desenvolvido um script em Python para realizar o carregamento dos dados das planilhas Excel para o banco de dados SQL Server.

O código faz basicamente três coisas principais:

- **Estabelece uma conexão entre o Python e o banco de dados SQL Server;**
- **Lê os arquivos Excel que contêm os dados do projeto;**
- **Insere esses dados nas tabelas staging criadas anteriormente.**

Dessa forma, o Python foi utilizado como ferramenta de integração entre os arquivos Excel e o banco de dados.

Explicação técnica:

O script utiliza duas bibliotecas principais.

Pandas

A biblioteca **Pandas** foi utilizada para ler os arquivos Excel e transformar esses dados em estruturas chamadas DataFrames, que funcionam como tabelas dentro do Python.

A função utilizada foi:

pd.read_excel()

Essa função permite carregar diretamente os dados das planilhas Excel.

SQLAlchemy

A biblioteca sqlalchemy foi utilizada para criar a conexão entre o Python e o SQL Server.

A função:

create_engine()

cria um mecanismo de conexão que permite ao Python enviar dados diretamente para o banco de dados.

A string de conexão define:

- O servidor SQL
- O banco de dados utilizado
- O driver ODBC
- O tipo de autenticação

Depois que os dados são carregados no Python, eles são enviados para o banco utilizando o comando:

to_sql()

Esse comando pega os dados armazenados nos DataFrames e insere diretamente nas tabelas do SQL Server.

O parâmetro:

if_exists="replace"

faz com que a tabela seja recriada caso já exista, garantindo que os dados sejam atualizados a cada execução do script.

Motivo da decisão

A decisão de utilizar Python para realizar o carregamento dos dados foi tomada para automatizar o processo de inserção das informações no banco de dados.

Como os dados estavam originalmente armazenados em planilhas Excel, seria possível inseri-los manualmente no SQL Server. No entanto, esse processo seria mais lento e menos escalável.

Utilizando Python, foi possível criar um pequeno processo de ingestão de dados que:

- **Lê os arquivos Excel automaticamente;**
- **Transforma esses dados em DataFrames;**
- **Envia os dados diretamente para o SQL Server.**

4.4 Criação das Tabelas Dimensionais e Fato

Código utilizado

```

CREATE TABLE dim_Banco (
    Banco_ID INT NOT NULL PRIMARY KEY,
    Banco NVARCHAR(50) NOT NULL
);

CREATE TABLE dim_Conta (
    Conta_ID INT NOT NULL PRIMARY KEY,
    Conta NVARCHAR(100) NOT NULL,
    Subgrupo NVARCHAR(100) NOT NULL
);

CREATE TABLE dim_Calendarario (
    Data DATE NOT NULL PRIMARY KEY,
    Ano INT NOT NULL,
    Mes VARCHAR(20) NOT NULL,
    MesAbrev VARCHAR(3) NOT NULL,
    MesNum INT NOT NULL,
    Dia INT NOT NULL
);

CREATE TABLE fato_Movimento (
    ID INT IDENTITY(1,1) PRIMARY KEY,
    Data DATE NOT NULL,
    Banco_ID INT NOT NULL,
    Conta_ID INT NOT NULL,
    Tipo CHAR(1) NOT NULL CHECK (Tipo IN ('E', 'S')),
    Valor DECIMAL(18,2) NOT NULL
);

CREATE TABLE fato_SaldoAnterior (
    Banco_ID INT NOT NULL PRIMARY KEY,
    Valor DECIMAL(18,2) NOT NULL
);

```

Explicação do que foi feito

Nesta etapa foram criadas as tabelas finais do banco de dados, que serão utilizadas para organizar os dados de forma adequada para análise.

Foram criados dois tipos de tabelas:

- **tabelas dimensionais (dim_), que armazenam informações descritivas**
- **tabelas fato (fato_), que armazenam os dados principais das movimentações**

As tabelas criadas foram:

- **dim_Banco**

- **dim_Conta**
- **dim_Calendario**
- **fato_Movimento**
- **fato_SaldoAnterior**

Explicação técnica

Tabelas dimensionais

As tabelas dimensionais foram criadas para armazenar informações mais descritivas.
Exemplo:

```
CREATE TABLE dim_Banco (
  Banco_ID INT NOT NULL PRIMARY KEY,
  Banco NVARCHAR(50) NOT NULL
);
```

- **PRIMARY KEY** → garante que cada banco seja único
- **NOT NULL** → impede valores vazios

A tabela dim_Conta segue a mesma lógica, armazenando informações de contas e subgrupos.

Tabela de calendário

A tabela dim_Calendario foi criada para trabalhar com datas de forma mais organizada.
Ela armazena:

- **data completa**
- **ano**
- **mês (nome e abreviação)**
- **número do mês**
- **dia**

Isso facilita análises por período, como por exemplo:

- **análise por mês**
- **comparação entre anos**

Tabelas fato

As tabelas fato armazenam os dados principais do projeto.
Exemplo:

```
CREATE TABLE fato_Movimento (
  ID INT IDENTITY(1,1) PRIMARY KEY,
  Data DATE NOT NULL,
  Banco_ID INT NOT NULL,
  Conta_ID INT NOT NULL,
  Tipo CHAR(1) NOT NULL CHECK (Tipo IN ('E', 'S')),
  Valor DECIMAL(18,2) NOT NULL
);
```

Aqui foram aplicados alguns conceitos importantes:

- **IDENTITY(1,1)** → cria um ID automático
- **CHECK (Tipo IN ('E','S'))** → garante que só existam entradas ou saídas

- **DECIMAL**(18,2) → usado para valores financeiros

Motivo das decisões

A decisão de separar as tabelas em dimensões e fatos foi tomada para organizar melhor os dados dentro do banco.

Se todos os dados ficassem em uma única tabela, haveria muita repetição de informações, como nomes de bancos e contas.

Ao separar:

- **As dimensões armazenam os dados descritivos**
- **As tabelas fato armazenam os valores principais**

isso reduz redundância e facilita a análise dos dados.

A criação da tabela de calendário também foi importante para permitir análises por data, algo comum em projetos de dados.

Mesmo sendo um projeto simples, essa estrutura segue um padrão bastante utilizado, o que contribui para o aprendizado.

4.5 Inserção dos Dados nas Tabelas

Código utilizado

```
INSERT INTO dim_Banco (Banco_ID, Banco)
SELECT DISTINCT Banco_ID, Banco
FROM stg_Bancos
WHERE Banco_ID IS NOT NULL AND Banco IS NOT NULL;

INSERT INTO dim_Conta (Conta_ID, Conta, Subgrupo)
SELECT DISTINCT Conta_ID, Conta, Subgrupo
FROM stg_Contas
WHERE Conta_ID IS NOT NULL;

INSERT INTO fato_Movimento (Data, Banco_ID, Conta_ID, Tipo, Valor)
SELECT
    CAST(m.Data AS DATE) AS Data,
    b.Banco_ID,
    c.Conta_ID,
    LEFT(m.Tipo, 1) AS Tipo,
    m.Valor
FROM stg_Movimentos m
INNER JOIN dim_Banco b ON m.Banco = b.Banco
INNER JOIN dim_Conta c ON m.Conta = c.Conta
WHERE m.Data IS NOT NULL;
);

INSERT INTO fato_SaldoAnterior (Banco_ID, Valor)
SELECT Banco_ID, Valor
FROM stg_SaldoAnterior
WHERE Banco_ID IS NOT NULL AND Valor IS NOT NULL;

ALTER TABLE fato_Movimento ADD FOREIGN KEY (Banco_ID) REFERENCES dim_Banco(Banco_ID);
ALTER TABLE fato_Movimento ADD FOREIGN KEY (Conta_ID) REFERENCES dim_Conta(Conta_ID);
ALTER TABLE fato_Movimento ADD FOREIGN KEY (Data) REFERENCES dim_Calendarario(Data);
ALTER TABLE fato_SaldoAnterior ADD FOREIGN KEY (Banco_ID) REFERENCES dim_Banco(Banco_ID);
```

Explicação do que foi feito

Nesta etapa foi realizada a inserção dos dados das tabelas staging nas tabelas finais do modelo, além da criação dos relacionamentos entre elas.

Os dados foram organizados nas seguintes tabelas:

- **dim_Banco**
- **dim_Conta**
- **fato_Movimento**
- **fato_SaldoAnterior**

Durante esse processo, os dados passaram por algumas validações, como remoção de valores nulos e eliminação de duplicidades.

Após a inserção dos dados, foram criados os relacionamentos entre as tabelas, garantindo a integridade das informações dentro do banco.

Explicação técnica

Inserção nas tabelas dimensionais

Foi utilizado o comando:

INSERT INTO ... SELECT ...

para inserir os dados a partir das tabelas staging.

Exemplo:

- **dim_Banco e dim_Conta recebem dados das tabelas stg_Bancos e stg_Contas.**

Foi utilizado DISTINCT para evitar duplicidade de registros.

Também foram aplicados filtros:

WHERE Banco_ID IS NOT NULL

para garantir que apenas dados válidos fossem inseridos.

Inserção na tabela fato

A tabela fato_Movimento foi preenchida a partir da tabela stg_Movimentos.

Durante essa etapa:

- **Foi utilizado CAST para ajustar o formato da data**
- **Foi utilizado LEFT() para padronizar o tipo de movimentação**
- **Foram utilizados JOINS para relacionar os dados com as tabelas dimensionais**

Exemplo:

INNER JOIN dim_Banco b ON m.Banco = b.Banco

INNER JOIN dim_Conta c ON m.Conta = c.Conta

Isso permite substituir nomes por identificadores (IDs), melhorando a organização dos dados.

Inserção do saldo anterior

A tabela **fato_SaldoAnterior** foi preenchida diretamente a partir da tabela **stg_SaldoAnterior**, com validação de valores nulos.

Criação dos relacionamentos

Foram criadas chaves estrangeiras (**FOREIGN KEY**) para garantir a integridade dos dados:

- **fato_Movimento → dim_Banco**

- **fato_Movimento** → **dim_Conta**
- **fato_Movimento** → **dim_Calendario**
- **fato_SaldoAnterior** → **dim_Banco**

Esses relacionamentos garantem que os dados estejam consistentes entre as tabelas.

Motivo das decisões

Essa etapa foi necessária para transformar os dados brutos das tabelas staging em um modelo organizado para análise.

A separação entre tabelas dimensionais e tabelas fato foi utilizada para:

- **evitar repetição de dados**
- **melhorar a organização**
- **facilitar consultas**

O uso de JOIN permitiu conectar os dados corretamente, enquanto os filtros ajudaram a manter a qualidade das informações.

A criação das chaves estrangeiras foi importante para garantir que não existam inconsistências entre os dados relacionados.

4.6 População da Tabela Calendário

Código utilizado

```

DECLARE @DataInicio DATE = '2023-01-01';
DECLARE @DataFim DATE = '2024-12-31';

WHILE @DataInicio <= @DataFim
BEGIN
    INSERT INTO dim_Calendario (Data, Ano, Mes, MesAbrev, MesNum, Dia)
    VALUES (
        @DataInicio,
        YEAR(@DataInicio),
        DATENAME(MONTH, @DataInicio),
        LEFT(DATENAME(MONTH, @DataInicio), 3),
        MONTH(@DataInicio),
        DAY(@DataInicio)
    );
    SET @DataInicio = DATEADD(DAY, 1, @DataInicio);
END;

```

Explicação do que foi feito

Nesta etapa foi realizada a população da tabela dim_Calendario, inserindo um conjunto completo de datas dentro de um intervalo definido.

Foi considerado o período de **01/01/2023 até 31/12/2024**, garantindo que todas as datas desse intervalo estejam disponíveis no banco de dados.

Além da data, também foram armazenadas informações complementares, como ano, mês, número do mês e dia, facilitando análises por período.

Explicação técnica

Para gerar as datas, foram utilizadas duas variáveis:

DECLARE @DataInicio DATE = '2023-01-01';

DECLARE @DataFim DATE = '2024-12-31';

Em seguida, foi utilizado um laço de repetição (WHILE) que percorre cada dia dentro desse intervalo:

WHILE @DataInicio <= @DataFim

A cada execução do loop:

- **A data atual é inserida na tabela**
- **São extraídas informações de tempo utilizando funções do SQL**

Funções utilizadas:

- **YEAR() → retorna o ano**
- **DATENAME(MONTH) → retorna o nome do mês**
- **LEFT() → cria a abreviação do mês**
- **MONTH() → retorna o número do mês**
- **DAY() → retorna o dia**

Após a inserção, a data é incrementada em um dia:

SET @DataInicio = DATEADD(DAY, 1, @DataInicio);

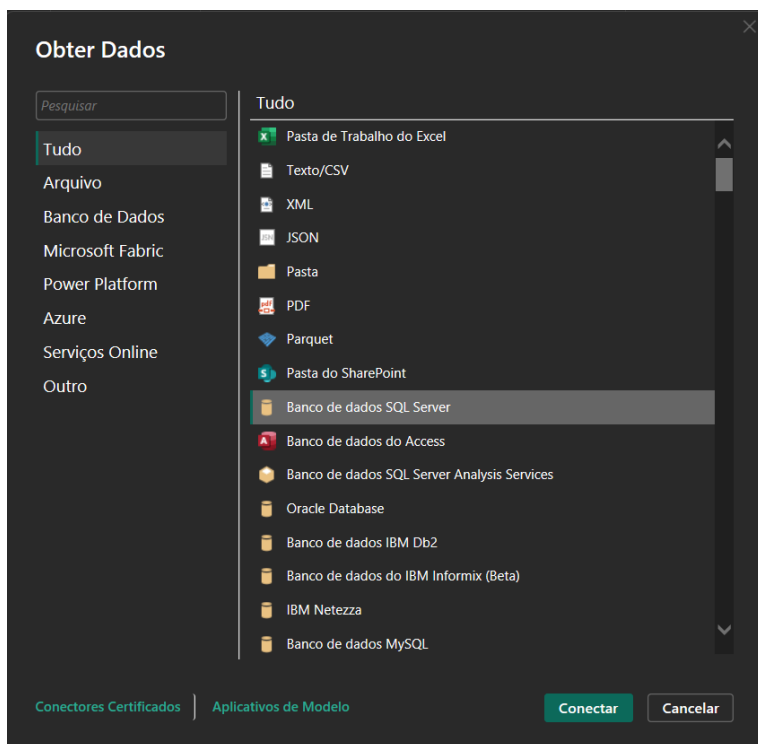
Motivo da decisão

A decisão de criar um calendário com intervalo fixo foi tomada para garantir que todas as datas estejam disponíveis para análise, mesmo que não existam movimentações em alguns dias.

Isso é importante porque evita problemas como:

- **falhas em gráficos no Power BI**
- **períodos sem dados aparentes**
- **dificuldade para análise contínua ao longo do tempo**

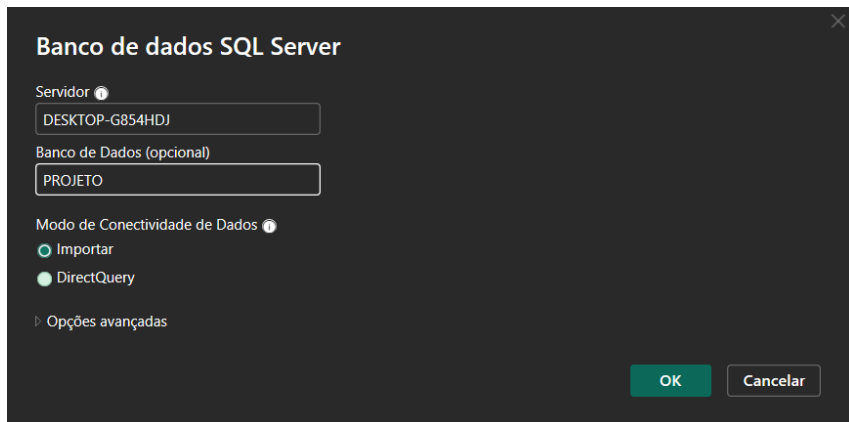
4.7 Conexão dos Dados no Power BI



Seleção da Fonte de Dados

O Power BI permite a conexão com diferentes fontes de dados.

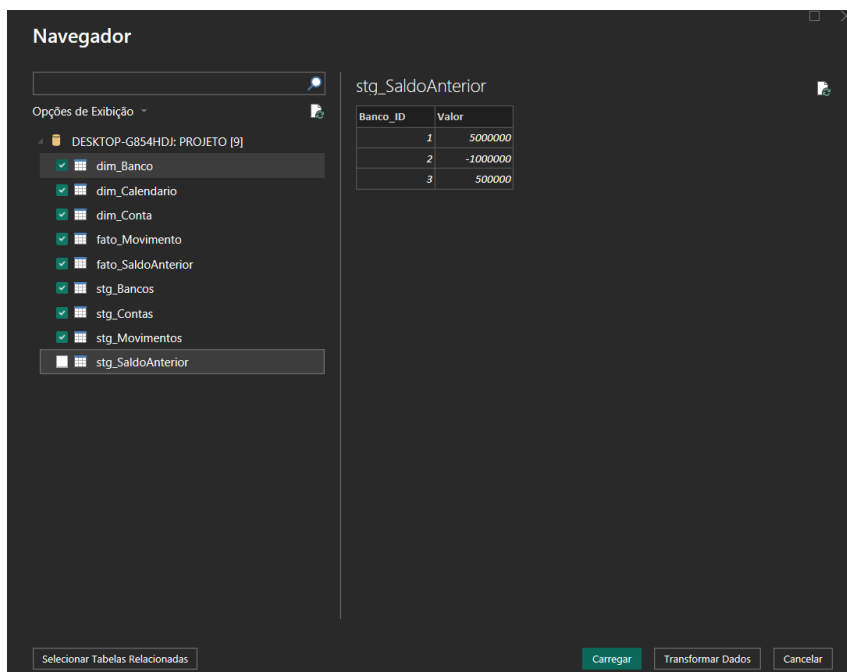
Neste projeto, foi utilizada a conexão com o SQL Server, pois os dados já estavam estruturados no banco de dados.



Configuração da Conexão

Após selecionar a fonte, foi realizada a configuração do servidor e do banco de dados utilizado no projeto.

Essa etapa permite que o Power BI acesse diretamente as tabelas armazenadas no SQL Server.



Configuração da Conexão

Após selecionar a fonte, foi realizada a configuração do servidor e do banco de dados utilizado no projeto.

Essa etapa permite que o Power BI acesse diretamente as tabelas armazenadas no SQL Server.

Explicação do que foi feito

Nesta etapa foi realizada a conexão do Power BI com o banco de dados SQL Server, permitindo importar as tabelas já organizadas para análise.

Foram selecionadas as tabelas dimensionais e fato, que haviam sido estruturadas previamente no banco de dados.

Motivo da decisão

A utilização do modo de importação foi escolhida por ser mais simples e adequado para o volume de dados do projeto.

Além disso, ao utilizar os dados já organizados no SQL Server, o Power BI pode focar apenas na análise e visualização, sem necessidade de tratamentos adicionais.

4.8 Tratamento e Uso dos Dados no Power BI

Explicação do que foi feito

Nesta etapa, os dados já estruturados no SQL Server foram importados para o Power BI, onde passaram a ser utilizados para análise e visualização.

Diferente da etapa inicial do projeto (quando os dados vinham diretamente do Excel), nesta fase o Power BI passa a trabalhar com dados já organizados e tratados no banco de dados.

O que muda no Power Query

No início do projeto, o Power Query era responsável por grande parte do tratamento dos dados, como:

- ajustes de colunas
- limpeza de dados

- **organização das informações**

Porém, nesta nova abordagem, essas transformações passaram a ser feitas no SQL. Com isso, o papel do Power Query se torna mais simples, sendo utilizado principalmente para:

- **carregar os dados do banco**
- **realizar pequenos ajustes, quando necessário**
- **preparar os dados para uso no modelo**

Essa mudança permite uma melhor separação das etapas do projeto:

- **SQL → responsável pela organização e transformação dos dados**
- **Power BI → responsável pela análise e visualização**

Motivo da decisão

Essa abordagem foi adotada para tornar o processo mais organizado e próximo de um cenário real de trabalho com dados.

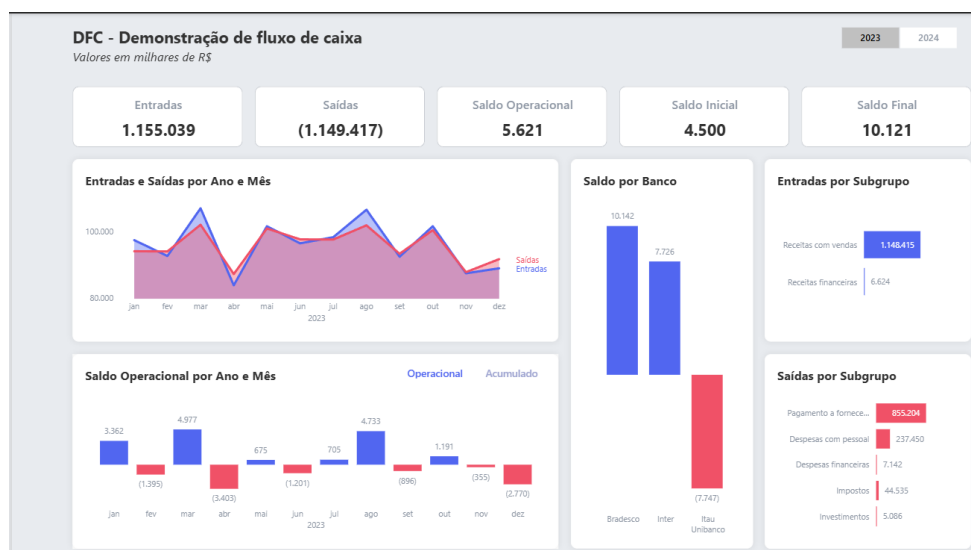
Ao centralizar o tratamento no banco de dados, o Power BI passa a trabalhar com dados mais limpos e estruturados, facilitando a criação dos relatórios.

4.9 Visualização Final dos Dados

O dashboard apresentado representa a etapa final do projeto, onde os dados organizados no banco de dados são utilizados para geração de informações visuais.

Nele, é possível visualizar de forma consolidada as entradas, saídas e saldos ao longo do tempo, além de análises por banco e por categoria.

Essa visualização foi construída a partir das tabelas estruturadas no SQL Server, demonstrando como os dados podem ser transformados em informações mais claras e acessíveis.



4.10 Dicionário de Dados

Explicação do que foi feito:

Nesta seção está registrada a estrutura completa de todas as tabelas criadas no banco de dados FluxoCaixaDW, organizadas por camada: staging, dimensional e fato. O dicionário documenta cada tabela com suas colunas, tipos de dados, restrições e descrição do conteúdo armazenado, servindo como referência técnica do modelo implementado.

Objetivo

Registrar de forma centralizada a definição de cada tabela do projeto, facilitando a consulta, manutenção e entendimento da estrutura de dados construída.

Tabelas Staging (stg_)

Área de entrada dos dados brutos carregados pelos arquivos Excel via Python.

stg_Bancos

Coluna	Tipo	NOT NULL	Restrição	Descrição
Banco_ID	INT	Não	—	Identificador numérico do banco
Banco	NVARCHAR(50)	Não	—	Nome do banco

stg_Contas

Coluna	Tipo	NOT NULL	Restrição	Descrição
Subgrupo_ID	INT	Não	—	Identificador numérico do subgrupo de conta
Subgrupo	NVARCHAR(100)	Não	—	Descrição do subgrupo
Conta_ID	INT	Não	—	Identificador numérico da conta
Conta	NVARCHAR(100)	Não	—	Descrição da conta

stg_Movimentos

Coluna	Tipo	NOT NULL	Restrição	Descrição
Tipo	NVARCHAR(20)	Não	—	Tipo da movimentação (entrada ou saída)
Conta	NVARCHAR(100)	Não	—	Nome da conta associada à movimentação
Data	DATETIME	Não	—	Data e hora da movimentação
Banco	NVARCHAR(50)	Não	—	Nome do banco da movimentação
Valor	DECIMAL(18,2)	Não	—	Valor monetário da movimentação

stg_SaldoAnterior

Coluna	Tipo	NOT NULL	Restrição	Descrição
Banco_ID	INT	Não	—	Identificador numérico do banco
Valor	DECIMAL(18,2)	Não	—	Saldo inicial do banco

Tabelas Dimensionais (dim_)

Armazenam dados descritivos utilizados como referência pelas tabelas fato. Possuem chave primária definida.

dim_Banco

Coluna	Tipo	NOT NULL	Restrição	Descrição
Banco_ID	INT	Não	PK	Identificador único do banco
Banco	NVARCHAR(50)	Não	—	Nome do banco

dim_Conta

Coluna	Tipo	NOT NULL	Restrição	Descrição
Conta_ID	INT	Não	PK	Identificador único da conta
Conta	NVARCHAR(100)	Não	—	Descrição da conta
Subgrupo	NVARCHAR(100)	Não	—	Subgrupo ao qual a conta pertence

dim_Calendarario

Coluna	Tipo	NOT NULL	Restrição	Descrição
Data	DATE	Não	PK	Data completa (chave primária)
Ano	INT	Não	—	Ano extraído da data
Mes	VARCHAR(20)	Não	—	Nome completo do mês
MesAbrev	VARCHAR(3)	Não	—	Abreviação do mês (3 letras)
MesNum	INT	Não	—	Número do mês (1 a 12)
Dia	INT	Não	—	Número do dia do mês

Tabelas Fato (fato_)

Armazenam os dados transacionais do projeto. Referenciam as dimensões por meio de chaves estrangeiras.

fato_Movimento

Coluna	Tipo	NOT NULL	Restrição	Descrição
ID	INT IDENTITY(1,1)	Não	PK	Identificador automático da movimentação
Data	DATE	Não	FK → dim_Calendarario	Data da movimentação
Banco_ID	INT	Não	FK → dim_Banco	Referência ao banco
Conta_ID	INT	Não	FK → dim_Conta	Referência à conta
Tipo	CHAR(1)	Não	CHECK('E','S')	E = Entrada, S = Saída
Valor	DECIMAL(18,2)	Não	—	Valor da movimentação

fato_SaldoAnterior

Coluna	Tipo	NOT NULL	Restrição	Descrição
Banco_ID	INT	Não	PK / FK → dim_Banco	Referência ao banco
Valor	DECIMAL(18,2)	Não	—	Saldo inicial do banco

5. Encerramento do Projeto

Considerações finais

Este projeto foi desenvolvido como uma evolução de um trabalho anterior, com o objetivo de aprender na prática como estruturar dados utilizando SQL e Python. A principal mudança foi sair de uma abordagem baseada em planilhas para uma estrutura mais organizada, utilizando banco de dados e automação.

Durante o desenvolvimento, foi possível compreender melhor como os dados podem ser organizados em etapas, desde a origem até a análise final.

Principais aprendizados

- **Criação e organização de tabelas em banco de dados**
- **Uso de SQL para transformação de dados**
- **Automação do processo de carga com Python**
- **Separação entre tratamento de dados e visualização**
- **Uso do Power BI com dados já estruturados**

Conclusão

De forma geral, o projeto cumpriu seu objetivo de demonstrar, na prática, como evoluir de uma solução simples para uma abordagem mais estruturada.

Essa experiência contribuiu para o entendimento de conceitos importantes da área de dados e representa um passo importante no aprendizado com SQL, Python e Power BI.